

Торговая платформа - Архитектура и детальное описание

🔗 Общее описание проекта

Торговая платформа - это высокопроизводительная система моделирования и визуализации торговли на финансовых рынках в реальном времени. Платформа демонстрирует современный подход к построению event-driven архитектуры с использованием шины событий (EventHub) как центрального компонента системы.

🏗 Архитектурная модель: EventHub-Центрическая Архитектура

Вершина архитектуры: EventHub (Шина событий)

EventHub - это центральный нервный узел всей системы, реализующий паттерн Publisher-Subscriber (Pub/Sub). Все компоненты общаются исключительно через события, что обеспечивает:

Ключевые характеристики EventHub:

1. Декомпозиция ответственности - каждый сервис знает только о событиях, а не о других сервисах
2. Масштабируемость - новые компоненты могут легко подключаться к системе
3. Отказоустойчивость - падение одного сервиса не приводит к остановке всей системы
4. Асинхронность - обработка событий происходит параллельно

Техническая реализация EventHub:

- Динамические подписки - компоненты могут подписываться на события во время выполнения
- Статические ключи - типизированные ключи событий для compile-time проверок
- Многоадресная рассылка - одно событие могут получать несколько подписчиков
- Каналы высокой пропускной способности (8192 сообщений в очереди)

csharp

// Пример регистрации события

```
[MessageKey("trade:executed")]  
public record TradeExecutedEvent(...) : MessageBase("trade:executed");
```

// Подписка на событие

```
_subscriptionManager.Subscribe<TradeExecutedEvent>(   
    "trade:executed",  
    async (tradeEvent, ct) => { /* обработка */ });
```

Сервис-Ориентированная Архитектура

1. QuotesGeneratorService - Сервис генерации котировок

Назначение: Имитация реального финансового рынка путем генерации случайных котировок.

Принцип работы:

- Каждые 2 секунды генерирует новые котировки для всех 24 тикеров
- Использует алгоритм `RanTable` из библиотеки `ChartDirector` для реалистичных данных
- Публикует события `QuoteGeneratedEvent` в `EventHub`

Взаимодействие:

-  Публикует: `QuoteGeneratedEvent`
-  Потребляет: ничего (самостоятельный генератор)
-  Цель: создать поток рыночных данных для тестирования стратегий

Технические детали:

- Генерация OHLCV данных (Open, High, Low, Close, Volume)
- Реалистичные временные метки (OLE Automation Dates)
- Контролируемая волатильность через параметры `minDelta/maxDelta`

2. StrategiesExecutionService - Сервис выполнения торговых стратегий

Назначение: Обработка котировок и выполнение торговых решений.

Принцип работы:

- Подписывается на `QuoteGeneratedEvent`
- Для каждой котировки находит соответствующий тикер
- Передаёт тик во все стратегии этого тикера
- Стратегии принимают решения о покупке/продаже

Взаимодействие:

-  Публикует: `TradeExecutedEvent`, `PositionChangedEvent`, `DealClosedEvent`, `OrderCreatedEvent`
-  Потребляет: `QuoteGeneratedEvent`
-  Цель: имитация работы реальных торговых алгоритмов

Ключевые особенности:

- 72 стратегии (24 тикера × 3 стратегии на тикер)
- Стратегии с разными периодами (5s, 10s, 15s)

- Изолированная обработка - сервис не блокирует генерацию данных

3. EventHubStrategy - Интеллектуальные торговые стратегии

Назначение: Принятие торговых решений на основе поступающих данных.

Принцип работы:

- Каждая стратегия работает независимо
- Анализирует поступающие тики
- Генерирует торговые сигналы (Buy/Sell/Hold)
- Управляет позицией (открытие/закрытие сделок)

Взаимодействие с EventHub:

```
csharp
// Все события стратегии публикуются в EventHub
this.OnTradeExecuted += OnTradeExecutedHandler;
this.OnDealClosed += OnDealClosedHandler;
this.OnPositionChanged += OnPositionChangedHandler;
this.OnOrderCreated += OnOrderCreatedHandler;
```

Типы стратегий:

- {Ticker}_05s - быстрая стратегия (5-секундный цикл)
- {Ticker}_10s - среднесрочная стратегия
- {Ticker}_15s - долгосрочная стратегия

4. InMemoryTradingDatabase - Сервис агрегации данных

Назначение: Централизованное хранение и агрегация торговых данных.

Основные функции:

- Хранение состояния - сделки, ордера, закрытые сделки, позиции
- Агрегация данных - расчет P&L, позиций, экспозиции
- Обновление UI - через события для SignalR
- Предоставление API - REST endpoints для фронтенда

Почему InMemory:

1. Производительность - операции в памяти на порядок быстрее
2. Простота - не требует внешних зависимостей (базы данных)
3. Достаточность - для демонстрационной платформы
4. Легкость сброса - перезапуск приложения = чистый старт

Структура данных:

- _trades - все выполненные сделки

- `_deals` - закрытые сделки с расчетом P&L
- `_orders` - ордера (открытые/закрытые)
- `_summaries` - агрегированные данные по тикеру+стратегии

5. QuotesConsoleService - Сервис мониторинга в реальном времени

Назначение: Визуализация рыночных данных в консоли.

Принцип работы:

- Подписывается на `QuoteGeneratedEvent`
- Форматирует и выводит котировки в консоль
- Использует цветовое кодирование для разных тикеров

Визуальные особенности:

- Цветные символы тикеров
- Стрелки направления движения цены (↑/↓)
- Процентное изменение
- Статистика производительности

6. TradingMonitorService - Сервис логирования и мониторинга

Назначение: Централизованное логирование важных событий.

Принцип работы:

- Подписывается на ключевые торговые события
- Логирует на уровне `Information` для важных действий
- Фильтрует шумовые события



Пользовательский интерфейс: Razor Pages + SignalR

Архитектура UI слоя

Razor Pages - Статический рендеринг

- Преимущества:
 - Быстрая первоначальная загрузка
 - SEO-дружелюбность
 - Простота разработки
- Используется для: Базовой структуры страниц, фильтров, пагинации

SignalR - Реальное время

- Назначение: Двусторонняя коммуникация в реальном времени
- Почему SignalR:
 1. WebSocket поверхность - автоматический fallback на Long Polling
 2. Масштабируемость - поддержка горизонтального масштабирования
 3. Группы и соединения - управление клиентскими подключениями
 4. Интеграция с .NET - родная поддержка в [ASP.NET](#) Core

Реализация реального времени в UI

1. TradingDataHub (SignalR Hub)

```
csharp
// События базы данных транслируются в SignalR
_database.SummaryUpdated += OnSummaryUpdated;
_database.TradeAdded += OnTradeAdded;

// Клиенты получают обновления в реальном времени
await Clients.All.SendAsync("UpdateSummary", summary);
```

2. JavaScript для автообновления

На каждой странице реализованы:

Основные функции:

- Автообновление (5-10 секундные интервалы)
- Динамическая загрузка данных через API
- Анимация обновления (fade эффекты)
- Подсветка новых записей (NEW бейджи)
- Динамическая статистика (пересчет на клиенте)

Пример для страницы Trades:

```
javascript
// Автообновление каждые 5 секунд
refreshInterval = setInterval(() => {
  loadTradesData();
}, 5000);

// Загрузка через API
async function loadTradesData() {
  const response = await fetch(`/api/trading/trades?ticker=${ticker}`);
  const data = await response.json();
  updateTradesTable(data.items);
}
```

3. Страницы и их особенности

Finance.cshtml - Финансовые графики

- Использует ChartDirector для профессиональных графиков
- Автообновление: 1.4 секунды через AJAX
- 24 графика по одному на каждый тикер
- Технические индикаторы: SMA, MACD, Stochastic, Donchian Channel

Summary.cshtml - Общая сводка

- Агрегированные данные по всем стратегиям
- Фильтрация по тикеру/стратегии
- Статистика: Total P&L, Open Positions, Avg P&L

Trades.cshtml - Сделки

- Все исполненные сделки в реальном времени
- Фильтрация и пагинация
- Подсветка новых сделок (за последние 10 секунд)

Orders.cshtml - Ордера

- История создания ордеров
- Статусы: Pending, Filled, Cancelled, Rejected
- Цветовое кодирование статусов

Deals.cshtml - Закрытые сделки

- Завершенные сделки с расчетом P&L
- Статистика: Profitable/Losing deals, Average P&L
- Расчет доходности в процентах

Positions.cshtml - Текущие позиции

- Открытые и закрытые позиции
- Расчет экспозиции и P&L%
- Фильтр "Only Open Positions"

4. Визуальные улучшения в UI

Цветовое кодирование:

- ☒ Зеленый: Прибыль, LONG позиции, Buy ордера
- ☒ Красный: Убыток, SHORT позиции, Sell ордера
- ☒ Желтый: Предупреждения, новые записи
- ☒ Синий: Информация, статистика

Интерактивные элементы:

- Автообновление с toggle кнопками
- Динамические фильтры без перезагрузки страницы

- Анимация обновления (fade эффекты)
- Подсветка изменений (новые/обновленные записи)

Статистика в реальном времени:

- Обновляемые карточки с ключевыми метриками
- Динамические графики (на странице Finance)
- Счетчики новых событий

Взаимодействие компонентов

Поток данных в системе:

text

QuotesGeneratorService

↓ (публикует QuoteGeneratedEvent)

EventHub

↓

StrategiesExecutionService (подписан на котировки)

↓ (публикует торговые события)

EventHub

↓

InMemoryTradingDatabase (подписан на торговые события)

↓ (агрегирует данные)

SignalR Hub ↔ Web UI

↓

Пользователь (видит обновления в реальном времени)

Event-driven цепочка:

text

1. Генерация котировки → EventHub
2. Стратегии обрабатывают → публикуют сделки → EventHub
3. База данных агрегирует → обновляет UI через SignalR
4. Пользователь видит изменения в реальном времени

Ключевые преимущества архитектуры

1. Гибкость и масштабируемость

- Новые сервисы можно добавлять без изменения существующего кода
- Каждый сервис может масштабироваться независимо
- EventHub служит буфером при пиковых нагрузках

2. Отказоустойчивость

- Падение одного сервиса не останавливает систему
- EventHub буферизует события до восстановления сервисов
- InMemory база может быть перезапущена без потери данных (в демо-режиме)

3. Производительность

- Асинхронная обработка событий
- InMemory хранение для минимальных задержек
- Пакетное обновление UI через SignalR

4. Тестируемость

- Каждый сервис можно тестировать изолированно
- EventHub позволяет мокать входные события
- InMemory база упрощает тестирование бизнес-логики

Технологический стек

- [ASP.NET](#) Core 6+ - бэкенд фреймворк
- Razor Pages - серверный рендеринг
- SignalR - реальное время
- ChartDirector - финансовые графики
- EventHub (кастомная реализация) - шина событий
- Bootstrap 5 - UI фреймворк
- JavaScript (ES6+) - клиентская логика
- InMemory Collections - хранение данных

Производительность системы

Обработка событий:

- 24 тикера × 3 стратегии = 72 стратегии
- Генерация котировок каждые 2 секунды
- До 36 событий в секунду только от котировок
 - - торговые события от стратегий

UI обновления:

- Автообновление: 5-10 секунд на бизнес-страницах

- **Графики: 1.4 секунды обновления**
- **Анимация без блокировки UI потока**

Возможности расширения

Потенциальные улучшения:

1. **Персистентность - сохранение данных в SQL/NoSQL базу**
2. **Кластеризация - горизонтальное масштабирование через Redis для SignalR**
3. **Авторизация - JWT tokens для защиты API**
4. **Мобильное приложение - через те же SignalR endpoints**
5. **AI/ML стратегии - интеграция моделей машинного обучения**
6. **Бэктестинг - историческое тестирование стратегий**
7. **Панель управления - админка для управления стратегиями**

Итог

Торговая платформа демонстрирует мощную, масштабируемую архитектуру, построенную вокруг EventHub как центрального компонента. Система сочетает в себе:

1. **Event-driven backend - все сервисы общаются через события**
2. **Real-time UI - мгновенные обновления через SignalR**
3. **Производительное хранение - InMemoryDataBase для минимальных задержек**
4. **Интуитивный интерфейс - современный, отзывчивый UI**

Эта архитектура не только эффективна для демонстрационной платформы, но и готова к масштабированию для production использования с добавлением персистентности, безопасности и дополнительных функциональностей.